Multi-Group-ACE
Technical Documentation

Multigroup ACE, MGXS HDF5, and Mesh Preparation Utilities

This document describes the code, example assets, and practical command-line workflows in the Multi-Group-ACE repository. It is intended as compact technical reference material for users working with multigroup cross sections, MCNP-style inputs, Gmsh meshes, and OpenSn or OpenMC compatible MGXS libraries.

March 22, 2026

Contents

1 Repository Layout 2

2 multi_group_ace/__init__.py and multi_group_ace/core.py 2

3 multi_group_ace/json_loader.py 3

4 multi_group_ace/converter.py 4

5 multi_group_ace/ace_io.py 5

6 multi_group_ace/hdf5_io.py 6

7 multi_group_ace/gmsh_geometry.py 8

8 multi_group_ace/cli.py 10

9 examples/ and json examples/ 11

10 examples/mg08_out/run_keigen.py 11

11 Workflows 12

1 Repository Layout

The repository is organized around a reusable Python package, a set of verification and example assets, and supporting documentation. Table 1 summarizes the main paths a new user will encounter.

Table 1: Top-level repository layout

Path	Role
`multi_group_ace/`	Primary package implementation. It contains the multigroup data model, JSON ingestion, ACE construction, ACE I/O, HDF5 MGXS output, MCNP-to-Gmsh mesh generation, and the main command-line driver.
`examples/`	Verification-style case directories. Each `mgXX` directory contains source inputs such as MCNP decks, descriptions, and ACE tables; matching `mgXX_out` directories contain generated assets such as `mgxs.h5`, `geom.msh`, and in one case an OpenSn execution driver.
`json examples/`	Sample JSON configuration files for the JSON to ACE workflow. These are useful as input templates for new multigroup material definitions.
`doc/`	Project documentation assets. This directory includes a PDF reference document and can also store generated technical documentation such as the present `.tex` file.
`notes/`	Internal project notes and short reference text files that capture usage context and development constraints.

The primary user-facing interfaces are `python -m multi_group_ace.cli` for package-based execution and `python examples/mg08_out/run_keigen.py` for a concrete OpenSn solve driven from Python.

2 `multi_group_ace/__init__.py` and `multi_group_ace/core.py`

Purpose

These two files define the package surface and the central data container used everywhere else in the repository. `core.py` provides the `MGData` class and physical constants for temperature conversion, while `__init__.py` re-exports the package entry points so downstream code can import from `multi_group_ace` directly instead of reaching into each module. In practice, they establish the common vocabulary of the codebase: every loader, converter, writer, and mesh generator either creates, consumes, or returns objects that are standardized here.

Inputs and Outputs

The main input to this logical module is a set of multigroup data values used to construct an `MGData` instance: material identifier, group structure, reaction cross sections, scattering matrices, temperature, and metadata. The main output is an in-memory object with a consistent layout, including the internal scattering convention of row equals destination group and column equals source group, plus a package namespace that exposes the loader, ACE builder, ACE I/O, HDF5

writer, and mesh-generation functions. If no anisotropic scattering moments are supplied, `MGData` automatically fills `sig1` with zeros so downstream code can always assume the field exists.

Key Dependencies

- Python standard library only.
- Downstream package modules re-exported through `__init__.py`.

Under the Hood

At import time, `__init__.py` executes a sequence of `from .module import name` statements. This has two effects. First, each imported object becomes available as an attribute on the package itself, so code can write `from multi_group_ace import build_ace` instead of importing from `converter.py` directly. Second, the `__all__` list defines which names are exported when users write `from multi_group_ace import *`; the file is therefore acting as a package-level facade rather than a place where new calculations occur.

`core.py` is intentionally passive. It does not validate input lengths, enforce units, or derive missing values; it only stores them. That design keeps the data model lightweight and lets validation remain close to the I/O boundary, such as the JSON loader or the ACE reader. The class also preserves the internal scatter ordering used throughout the package:

```
row = destination group
column = source group

For NG = 2:
[1<-1, 2<-1, 1<-2, 2<-2]
```

This convention matters because several external formats use different flattening rules. The converter and HDF5 writer both rely on the same `MGData` layout and explicitly reorder the data only at the point where an external file format requires a different convention. The constants `KB_EV_PER_K` and `MEV_TO_EV` are also defined here so temperature conversion is numerically consistent between ACE, HDF5, and OpenSn-facing routines.

Command-Line Usage

```
python -c 'from multi_group_ace import load_from_json; mg, _ = load_from_json("config/
    two_group_problem.json"); print(mg.zaid, mg.NG)'
```

3 multi_group_ace/json_loader.py

Purpose

This module reads a JSON description of multigroup data and converts it into an `MGData` object. It validates the expected keys, enforces a strictly descending energy grid, handles optional microscopic-to-macroscopic conversion using atom density, normalizes fission spectrum data when needed, and assigns default metadata such as the output file stem and date.

Inputs and Outputs

The input is a JSON file containing fields such as `zaid`, `NG`, `erg`, `sigt`, `sigc`, `sigf`, `nu`, `chi`, and `sigs`, with optional fields including `sig1`, `is_micro`, `atom_density`, `nbins`, `comment`, `tmp`, and `file`. The output is a tuple containing a populated `MGData` object and the base filename that should be used for the generated ACE table.

Key Dependencies

- `json`
- `datetime`
- `multi_group_ace.core`

Parsing and Validation Flow

The loader begins by reading the entire JSON file into a Python dictionary and immediately checking for the required keys. It then extracts the ZA portion from `zaid`, converts the energy grid to floating point values, and passes the grid through a helper that reverses it if necessary and rejects any non-strict ordering. Scalar vectors such as `sigt`, `sigc`, and `sigf` are expanded to exactly `NG` entries, while scattering data can be supplied either as a flat array or as an `NG x NG` nested list.

If `is_micro` is true, the loader multiplies the supplied microscopic cross sections by `atom_density` before creating `MGData`. If fission is absent, it explicitly zeros out `sigf`, `nu`, and `chi`; if fission is present but `chi` is not normalized, it normalizes the spectrum so the HDF5 and ACE writers later receive a consistent source vector. The loader also interprets `tmp > 1.0` as a temperature-like quantity that should be converted into ACE-style `kT` in MeV.

Command-Line Usage

```
python -c 'from multi_group_ace.json_loader import load_from_json; mg, out =
    load_from_json("json examples/ex4.json"); print(out, mg.erg)'
```

4 multi_group_ace/converter.py

Purpose

This module converts the abstract multigroup representation stored in `MGData` into the `NXS`, `JXS`, and `XSS` arrays expected by a simple multigroup ACE table. It also contains the equiprobable angular-bin generator used when anisotropic scattering is present and the P1 moment must be represented through angular bin edges.

Inputs and Outputs

The input is one `MGData` instance containing energy boundaries, total, capture, fission, and scattering data, plus optional P1 scattering moments and an angular bin count. The output is a three-part ACE representation: integer control arrays `NXS` and `JXS`, and the floating-point payload array `XSS`, all ready to be written to disk by the ACE I/O module.

Key Dependencies

- `multi_group_ace.core`

How the ACE Arrays Are Built

The converter first determines whether the material should be treated as isotropic by checking whether every entry in `sig1` is zero. It then fills the `NXS` control array with global table metadata such as the ZA identifier, group count, and anisotropy flag, and it fills the `JXS` array with pointers into the `XSS` payload. In other words, `NXS` describes what kind of table this is, `JXS` describes where each block begins, and `XSS` stores the actual floating-point data.

The energy grid is stored in ACE as midpoint-width pairs rather than direct group boundaries, so the converter computes a midpoint and width for each group from the descending energy edges in `mg.erg`. Reaction vectors such as total, fission, `nu`, `chi`, and capture are then written into consecutive `XSS` blocks. The scattering matrix is transposed at this stage because the package stores scattering as destination-by-source, while the ACE block is written source-by-destination.

If anisotropic scattering is present, the converter allocates additional pointer blocks and writes equiprobable angular bin edges for each nonzero scattering pair. Those angular edges are not stored as a direct P1 moment; they are stored as a sequence of cosine boundaries that reproduce the desired first angular moment when sampled uniformly within each bin. This is why the ACE reader later has to reconstruct an approximate `sig1` from the bin edges rather than reading `sig1` directly from a dedicated array.

Command-Line Usage

```
python -c 'from multi_group_ace.json_loader import load_from_json; from multi_group_ace.
    converter import build_ace; mg, _ = load_from_json("config/two_group_problem.json");
    nxs, jxs, xss = build_ace(mg); print(nxs[1], jxs[13], len(xss) - 1)'
```

5 multi_group_ace/ace_io.py

Purpose

This module reads and writes simple multigroup ACE files and related text outputs. It reconstructs `MGData` from an ACE table, writes ACE text files, emits an `xsdir` entry suitable for MCNP-style use, and produces a compact summary report of the groupwise reaction data and scattering totals. It is the part of the package that translates between the package's internal data model and the pointer-based ACE table layout found on disk.

Inputs and Outputs

Inputs include an existing ACE file path for reading or, for writing, an `MGData` object together with the `NXS`, `JXS`, and `XSS` arrays produced by the converter. Outputs include an ACE table file, a summary text file with per-group totals, and an `xsdir` text file that can be referenced from MCNP input.

Key Dependencies

- Python standard library file I/O
- `multi_group_ace.core`

How ACE Parsing Works

The reader treats the ACE file as a structured text record with four layers. It first reads the two header lines, then skips the four fixed-format lines that the writer emits as placeholders, then reads the 16 `NXS` integers and 32 `JXS` integers, and finally reads exactly `NXS(1)` floating-point values into the `XSS` payload array. The arrays are stored internally as one-based lists so the code can mirror the indexing style used by ACE documentation and by the original table structure.

Once the control arrays are loaded, the reader knows how to interpret the payload. It uses `NXS(5)` to determine the number of energy groups, `NXS(2)` to recover the ZA value, and the pointer `JXS(1)` to find the midpoint-width representation of the energy grid. The code reconstructs the descending group boundaries by computing

$$E_{\text{high}} = \text{mid} + \tfrac{1}{2}\text{width}, \qquad E_{\text{low}} = \text{mid} - \tfrac{1}{2}\text{width}.$$

It then reads the total, fission, `nu`, `chi`, and capture vectors from the blocks pointed to by `JXS(2)` through `JXS(6)`.

Scattering is handled in two stages. First, the reader locates the P0 scattering block by following the pointer stored at `JXS(13)`, then transposes the ACE ordering into the package ordering so that `mg.sigs` again means destination-by-source. Second, it determines whether angular information is present by checking `NXS(3)`. If `NXS(3) <= 0`, the table is treated as isotropic and the returned `sig1` array is all zeros. If `NXS(3) > 0`, the file contains equiprobable angular bins; the reader uses `JXS(16)` and `JXS(17)` to find the location-offset table and the cosine-edge table, computes the average cosine in each populated scattering bin, and multiplies that by the P0 scattering value to recover an approximate P1 moment. This is how the code distinguishes isotropic from anisotropic tables and reconstructs enough angular information for downstream OpenSn/OpenMC-style output.

The writer performs the inverse operation. It does not infer new physics from the arrays; it serializes the `MGData`-derived `NXS`, `JXS`, and `XSS` arrays back into ACE text format, preserving the same pointer layout. The auxiliary writers then produce an `xsdir` entry and a summary table so the table can be inspected or referenced by external transport inputs.

Command-Line Usage

```
python -c 'from multi_group_ace.ace_io import read_mg_ace; mg, _, _, _ = read_mg_ace("ace
    /problem.ace"); print(mg.zaid, mg.comment_short, mg.NG)'
```

6 multi_group_ace/hdf5_io.py

Purpose

This module writes multigroup cross-section libraries in an OpenMC-compatible HDF5 structure that OpenSn can also consume. It handles temperature naming, fixed-length ASCII attributes, energy-group metadata, per-nuclide datasets, and both single-nuclide and multi-nuclide library

output. The key responsibility of the module is to convert the package's in-memory representation into the exact hierarchy of groups, datasets, and attributes expected by downstream MGXS readers.

Inputs and Outputs

Inputs are one or more `MGData` objects plus an output `.h5` filename and, optionally, a library name to use inside the HDF5 file. The output is an `mgxs.h5` file containing top-level library metadata, an energy-group structure stored in ascending eV, and per-material groups with total, absorption, fission, chi, nu-fission, and scatter data laid out in the OpenMC multigroup schema. The module writes either a single nuclide group or one group per ACE-derived material, but in both cases the root-level structure of the file is the same.

Key Dependencies

- `h5py`
- `multi_group_ace.core`

HDF5 Structure Written by the Code

At the root of the file, the writer creates a small set of attributes that describe the library as a whole. The string attributes are written with fixed-length ASCII datatypes because some OpenSn readers expect C-style fixed-size strings rather than variable-length UTF strings. The root then stores the global group structure in ascending eV, even though `MGData` stores energy boundaries internally in descending MeV.

```
/
  attrs:
    filetype = "mgxs"
    version = [1, 0]
    energy_groups = NG
    delayed_groups = 0
    group structure = [E0, E1, ..., ENG]   # ascending, in eV

  /<nuclide_name>
    attrs:
      atomic_weight_ratio = AWR
      fissionable = true/false
      representation = "isotropic" or "angle"
      scatter_format = "legendre"
      scatter_shape = "[G][G'][Order]"
      order = 0 or 1

    /kTs
      /294K = kT in eV

    /294K
      /total
      /absorption
      /fission            # only if fissionable
      /chi                # only if fissionable
      /nu-fission         # only if fissionable
      /scatter_data
```

```
        /g_min
        /g_max
        /scatter_matrix
        /multiplicity_matrix
```

How the Data Are Mapped into That Structure

The temperature group name is created by converting ACE-style `kT` in MeV into eV and then into Kelvin using the Boltzmann constant from `core.py`. For a room-temperature table, this typically produces a group named `294K`. The writer also computes `absorption` as `sigc + sigf`, which matches the way transport codes often expect absorptive loss to be reported independently from scattering.

Scattering is where most of the format conversion occurs. The helper `_scatter_order()` examines `mg.sig1`; if every P1 term is zero, the file is written as order 0 with `representation = "isotropic"`. If any P1 term is nonzero, the file is written as order 1 with `representation = "angle"`. The helper `_scatter_matrix_openmc_order()` then flattens the internal destination-by-source matrix into the ordering expected by the HDF5 schema: incoming group outer loop, outgoing group inner loop, and, for anisotropic data, P0 followed by P1 for each pair. The datasets `g_min` and `g_max` are filled densely, which indicates that each incoming group potentially scatters across the full group range represented by the stored matrix.

When writing a combined problem library, `write_problem_mgxs_library()` first checks that every material shares the same group count and the same energy grid. It then writes one nuclide group per `MGData` object beneath the same root. This makes the resulting `mgxs.h5` file suitable for problems in which a single mesh uses multiple material datasets.

Command-Line Usage

```
python -m multi_group_ace.cli --ace ace/problem.ace --h5 out/problem_mgxs.h5
    --library-name 99902.02m
```

7 multi_group_ace/gmsh_geometry.py

Purpose

This module converts available geometry descriptions into a Gmsh mesh file that can be paired with the generated cross sections. It prefers explicit `.geo` or `.step`/`.stp` geometry files when they exist, and otherwise parses a subset of MCNP surface cards such as planes, spheres, and cylinders to build a corresponding mesh through the Gmsh Python API. It therefore serves as both a geometry selection layer and a geometry translation layer.

Inputs and Outputs

Inputs are a source problem directory or an individual MCNP input file together with an output mesh path and, optionally, a Gmsh executable path. Outputs are `geom.msh` files with physical region tags and deterministic axis-aligned boundary names such as `xmin`, `xmax`, `ymin`, and `zmax`, which makes later boundary-condition assignment more predictable. When a directory contains an explicit `.geo` or `.step`/`.stp` file, that geometry is meshed directly; otherwise the code derives geometry from the text of the MCNP input.

Key Dependencies

- `os`
- `subprocess`
- `gmsh` Python package or a local Gmsh executable

From MCNP Text to Geometry Objects

The fallback path begins in `_parse_mcnp_surfaces()`, which reads the MCNP input file line by line as plain text. For each line, it removes any inline comment that starts with `$`, skips blank lines, skips comment cards beginning with `c`, and ignores any line whose first token is not a numeric surface identifier. The second token is then treated as the surface mnemonic and normalized, so forms such as `c/z` are converted to `cz` before interpretation.

Only a subset of MCNP surface cards is recognized, but each supported card is translated into a lightweight Python geometry object:

- `px`, `py`, `pz` become `Plane` objects.
- `so`, `s`, and `sph` become `SphereSurf` objects.
- `cz`, `cx`, `cy`, and `rcc` become `CylinderSurf` objects.

The parser therefore does not build a general constructive solid geometry model of all MCNP syntax. Instead, it extracts enough structured information to build a limited class of slab, spherical, cylindrical, and simple infinite-domain proxy meshes.

How Those Objects Become a Gmsh Mesh

After parsing, `mcnp_to_gmsh_mesh()` decides which builder to use. If spheres are present, it creates nested Gmsh spheres and subtracts each inner sphere from the next outer sphere to form shells. If cylinders are present, it creates one or more cylinders and similarly subtracts nested radii to form cylindrical shells or rings. If plane surfaces are present, it sorts unique cut locations and creates adjacent boxes between consecutive cuts; when multiple slab regions exist, it uses Gmsh fragmentation so the internal interfaces are conformal rather than duplicated coincident faces.

The filename suffixes `-IN`, `-SL`, `-CY`, `-SP`, and `-ISLC` are used only as hints. They help the code choose a fallback geometric interpretation when the parsed primitive lists are sparse or ambiguous, but they do not replace parsing. Once regions have been created, each 3D volume is assigned a physical group name such as `region_1`, `region_2`, and so on. The helper `_tag_axis_aligned_boundary_surfaces()` then scans the mesh boundary surfaces, computes bounding boxes, and tags outer faces as `xmin`, `xmax`, `ymin`, `ymax`, `zmin`, or `zmax` when they are axis-aligned.

Finally, Gmsh generates a three-dimensional mesh and writes it to disk. After writing, the file is normalized to Unix line endings because OpenSn expects text mesh files with `LF` newlines. When `make_mesh_for_problem()` finds a `.geo` or `.step` file instead of relying on MCNP parsing, it bypasses the Python geometry builders entirely and invokes the Gmsh executable directly on that source file.

Command-Line Usage

```
python -c 'from multi_group_ace.gmsh_geometry import make_mesh_for_problem;
    make_mesh_for_problem("cases/sphere_case", "out/sphere_case", gmsh_exe="gmsh")'
```

8 multi_group_ace/cli.py

Purpose

This module is the main command-line entry point for the package. It exposes three user-facing workflows: JSON to ACE generation, ACE to HDF5 conversion, and recursive directory scanning that builds one `mgxs.h5` file per problem directory while optionally generating a companion mesh.

Inputs and Outputs

The CLI accepts `-config` for JSON-driven ACE generation, `-ace` and `-h5` for single-file HDF5 conversion, or `-verification-root` and `-outdir` for recursive batch processing. Depending on the mode, it produces ACE tables, `xsdir` and summary text files, one or more `mgxs.h5` libraries, and optional `geom.msh` meshes.

Key Dependencies

- `argparse`
- `os`
- `multi_group_ace.json_loader`
- `multi_group_ace.converter`
- `multi_group_ace.ace_io`
- `multi_group_ace.hdf5_io`
- `multi_group_ace.gmsh_geometry`

Dispatch Logic

The CLI uses mutually exclusive high-level modes selected by which arguments are present. If `-verification-root` is supplied, it walks the directory tree, tries to interpret every file as an ACE table, groups successful reads by output problem directory, and writes one combined `mgxs.h5` per problem. If `-ace` is supplied, it performs a single ACE-to-HDF5 conversion. If neither of those is present, it expects `-config` and runs the JSON-to-ACE workflow, including summary and `xsdir` generation.

The batch mode is particularly important because it is where the repository turns many individual ACE files into a problem-oriented asset layout. Instead of writing one HDF5 file per ACE file, it aggregates all ACE tables discovered under the same source problem directory and emits a single library containing one HDF5 group per material. That grouping matches the way a transport problem will later assign multiple materials to one mesh.

Command-Line Usage

```
python -m multi_group_ace.cli --verification-root examples --outdir build/verification
    --make-mesh --gmsh gmsh
```

9 examples/ and json examples/

Purpose

These directories provide concrete test assets for the codebase. The `examples/` tree contains source problem directories and generated outputs for several verification-style cases, while `json examples/` supplies compact JSON inputs for defining new multigroup materials without starting from an ACE file.

Inputs and Outputs

Typical inputs in `examples/mgXX/` include MCNP input decks, a `description.json` file, and one or more ACE tables. Typical outputs in `examples/mgXX_out/` include `mgxs.h5`, `geom.msh`, and, for one case, a Python OpenSn driver plus solver outputs such as `k_eff.txt` and ParaView files.

Key Dependencies

- No direct Python dependencies; these directories are data and example assets consumed by the CLI and example scripts.

Command-Line Usage

```
python -m multi_group_ace.cli --config "json examples/ex4.json"
```

10 examples/mg08_out/run_keigen.py

Purpose

This script is a Python-based OpenSn execution driver that demonstrates how to use the generated mesh and MGXS library in a transport solve. It loads `geom.msh` and `mgxs.h5`, constructs a discrete ordinates problem with a product quadrature, assigns vacuum boundary conditions, executes a power-iteration k-eigenvalue solve, and writes field-function output for visualization.

Inputs and Outputs

The required inputs are a Gmsh mesh file, an `mgxs.h5` library, a dataset name inside that library, and a temperature value used to select the correct multigroup data. Outputs include the reported eigenvalue, mesh export files, and scalar flux files in ParaView-compatible `.pvtu` form. This repository does not currently generate a standalone OpenSn text input deck; the OpenSn-facing execution example is this Python script.

Key Dependencies

- `pyopensn`
- `os`
- `re`

Command-Line Usage

```
python examples/mg08_out/run_keigen.py
```

11 Workflows

Workflow 1: JSON Material Definition to ACE Output

This workflow starts from a compact JSON description of groupwise cross sections and produces a simple multigroup ACE table together with a summary text file and an `xsdir` entry. It is the shortest path when the source data already exist in grouped form and the immediate target is an MCNP-style ACE table.

```
python -m multi_group_ace.cli --config "json examples/ex4.json"
python -c 'from multi_group_ace.ace_io import read_mg_ace; mg, _, _, _ = read_mg_ace("
    mg02xsecs"); print(mg.zaid, mg.NG)'
```

Workflow 2: ACE Library and MCNP Geometry to OpenSn-Ready Assets

This workflow combines ACE-to-HDF5 conversion with MCNP geometry-to-Gmsh conversion. The result is a pair of artifacts, `mgxs.h5` and `geom.msh`, that can be consumed by downstream OpenSn or OpenMC-style multigroup workflows.

```
python -m multi_group_ace.cli --ace ace/problem.ace --h5 out/problem/mgxs.h5
    --library-name 99902.02m
python -c 'from multi_group_ace.gmsh_geometry import make_mesh_for_problem;
    make_mesh_for_problem("cases/problem_case", "out/problem", gmsh_exe="gmsh")'
```

Workflow 3: Batch Verification Conversion with Mesh Generation

This workflow scans a directory tree of verification problems, identifies valid ACE tables, groups them by problem directory, writes one combined `mgxs.h5` per problem, and optionally generates a mesh next to each library. It is the most efficient way to prepare a large set of bundled examples for solver-side inspection.

```
python -m multi_group_ace.cli --verification-root examples --outdir build/verification
    --make-mesh --gmsh gmsh
```

Workflow 4: Prepared MGXS and Mesh to OpenSn Solve

This workflow shows the full handoff from repository-generated assets into an OpenSn calculation. It first creates the cross-section library and mesh for a case directory, then runs the Python OpenSn driver that loads those files, solves the k-eigenvalue problem, and exports visualization data.

```
python -m multi_group_ace.cli --ace examples/mg08/99902.02m --h5 examples/mg08_out/mgxs.
    h5 --library-name 99902.02m
python -c 'from multi_group_ace.gmsh_geometry import make_mesh_for_problem;
    make_mesh_for_problem("examples/mg08", "examples/mg08_out", gmsh_exe="gmsh")'
python examples/mg08_out/run_keigen.py
```